

SDN-Based Network Traffic Monitoring & Visualization System

What is SDN?

Unlike traditional network which has control plane for every switch , sdn has a centralized sdn controller.

It has 3 layers.

- **Application Layer:** It contains the typical network applications like intrusion detection, firewall, and load balancing. We use python + Grafana in our project
 - *North bound interface:* rest API
- **Control Layer:** It consists of the SDN controller which acts as the brain of the network. It also allows hardware abstraction to the applications written on top of it. We use open daylight here
 - *South bound interface:* Open flow
- **Infrastructure Layer:** This consists of physical switches which form the data plane and carries out the actual movement of data packets. Mininet switches to simulate

Objective: To design and implement a centralized network monitoring system using Software-Defined Networking (SDN) principles. The system decouples the control plane from the data plane to monitor network flows, collect real-time traffic statistics, and visualize them for anomaly detection.

Architecture Components:

- **Infrastructure Layer:** Mininet (Virtual Network Emulator)
- **Control Layer:** OpenDaylight (SDN Controller)
- **Application Layer:** Python Middleware (Data Collector)

- **Storage & Visualization:** Prometheus (Time-Series Database) & Grafana (Dashboard)

2. Step-by-Step Implementation Process

Phase 1: Environment Setup & Controller Installation The first phase involved setting up the SDN controller, which acts as the "brain" of the network.

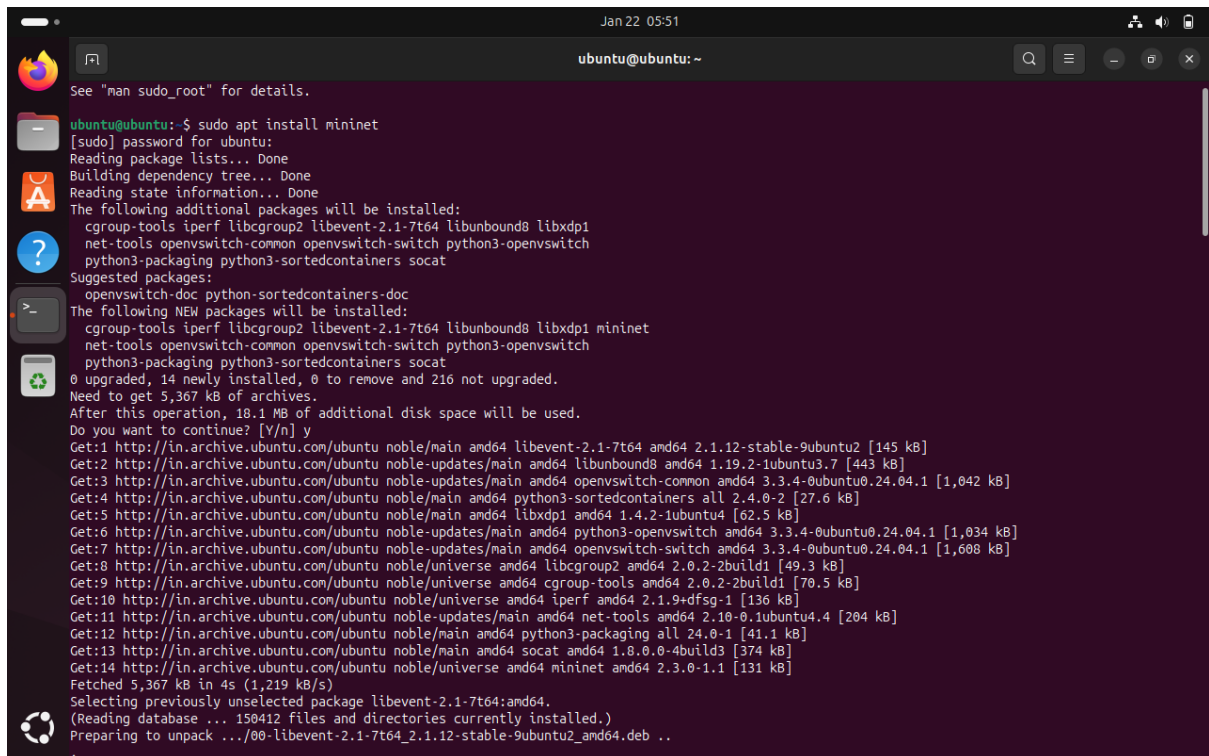
1. **System Update:** We started by updating the Ubuntu VM repositories to ensure compatibility.

Command: `sudo apt update && sudo apt upgrade -y`

Mininet:

Installing Mininet: We installed Mininet to simulate the network infrastructure (switches and hosts) without needing physical hardware.

- **Command:** `sudo apt install mininet -y`



```

Jan 22 05:51
ubuntu@ubuntu: ~
See "man sudo_root" for details.
ubuntu@ubuntu:~$ sudo apt install mininet
[sudo] password for ubuntu:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  cgroup-tools iperf libcgroup2 libevent-2.1-7t64 libunbound8 libxdp1
  net-tools openvswitch-common openvswitch-switch python3-openvswitch
  python3-packaging python3-sortedcontainers socat
Suggested packages:
  openvswitch-doc python-sortedcontainers-doc
The following NEW packages will be installed:
  cgroup-tools iperf libcgroup2 libevent-2.1-7t64 libunbound8 libxdp1 mininet
  net-tools openvswitch-common openvswitch-switch python3-openvswitch
  python3-packaging python3-sortedcontainers socat
0 upgraded, 14 newly installed, 0 to remove and 216 not upgraded.
Need to get 5,367 kB of archives.
After this operation, 18.1 MB of additional disk space will be used.
Do you want to continue? [Y/n] y
Get:1 http://in.archive.ubuntu.com/ubuntu noble/main amd64 libevent-2.1-7t64 amd64 2.1.12-stable-9ubuntu2 [145 kB]
Get:2 http://in.archive.ubuntu.com/ubuntu noble-updates/main amd64 libunbound8 amd64 1.19.2-1ubuntu3.7 [443 kB]
Get:3 http://in.archive.ubuntu.com/ubuntu noble-updates/main amd64 openvswitch-common amd64 3.3.4-0ubuntu0.24.04.1 [1,042 kB]
Get:4 http://in.archive.ubuntu.com/ubuntu noble/main amd64 python3-sortedcontainers all 2.4.0-2 [27.6 kB]
Get:5 http://in.archive.ubuntu.com/ubuntu noble/main amd64 libxdp1 amd64 1.4.2-1ubuntu4 [62.5 kB]
Get:6 http://in.archive.ubuntu.com/ubuntu noble-updates/main amd64 python3-openvswitch amd64 3.3.4-0ubuntu0.24.04.1 [1,034 kB]
Get:7 http://in.archive.ubuntu.com/ubuntu noble-updates/main amd64 openvswitch-switch amd64 3.3.4-0ubuntu0.24.04.1 [1,608 kB]
Get:8 http://in.archive.ubuntu.com/ubuntu noble/universe amd64 libcgroup2 amd64 2.0.2-2build1 [49.3 kB]
Get:9 http://in.archive.ubuntu.com/ubuntu noble/universe amd64 cgroup-tools amd64 2.0.2-2build1 [70.5 kB]
Get:10 http://in.archive.ubuntu.com/ubuntu noble/universe amd64 iperf amd64 2.1.0-1dfsg-1 [136 kB]
Get:11 http://in.archive.ubuntu.com/ubuntu noble-updates/main amd64 net-tools amd64 2.10-0.1ubuntu4.4 [204 kB]
Get:12 http://in.archive.ubuntu.com/ubuntu noble/main amd64 python3-packaging all 24.0-1 [41.1 kB]
Get:13 http://in.archive.ubuntu.com/ubuntu noble/main amd64 socat amd64 1.8.0.0-4build3 [374 kB]
Get:14 http://in.archive.ubuntu.com/ubuntu noble/universe amd64 mininet amd64 2.3.0-1.1 [131 kB]
Fetched 5,367 kB in 4s (1,219 kB/s)
Selecting previously unselected package libevent-2.1-7t64:amd64.
(Reading database ... 150412 files and directories currently installed.)
Preparing to unpack .../00-libevent-2.1-7t64_2.1.12-stable-9ubuntu2_amd64.deb ..

```

Verifying Mininet: We ran a built-in test to confirm Mininet was installed correctly and could create a basic network.

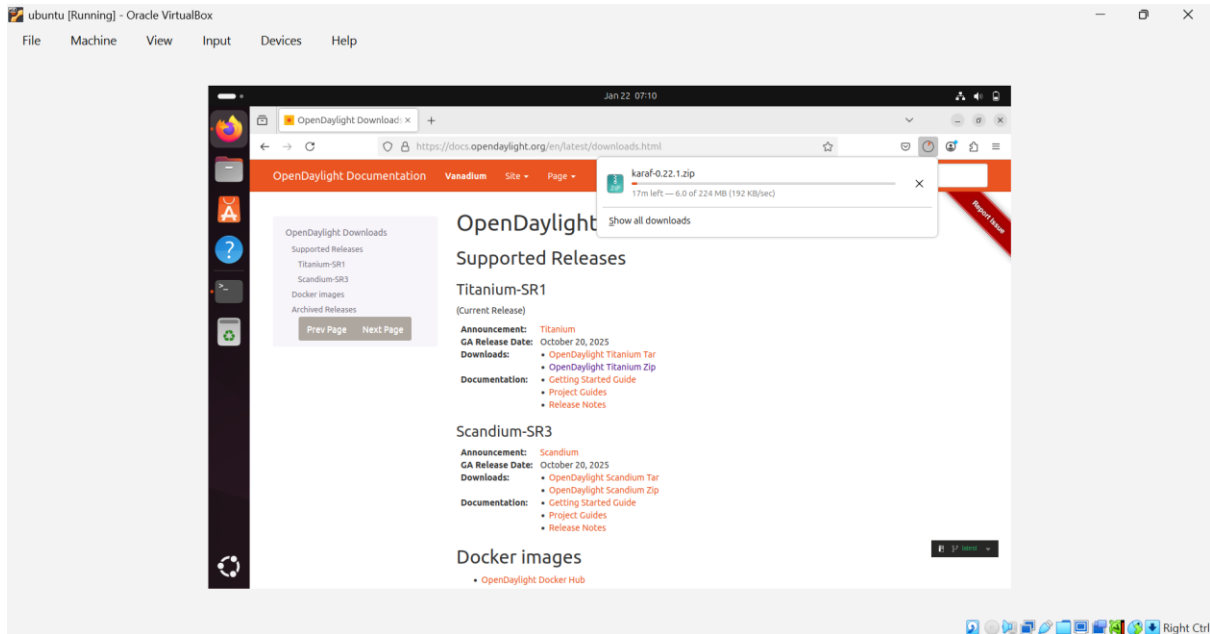
- **Command:** `sudo mn --test pingall`

```
Jan 22 05:49
ubuntu@ubuntu: ~
(type mn -h for details)
mn: error: option -t: invalid integer value: 'est'
ubuntu@ubuntu:~$ sudo mn --test ping all
Usage: mn [options]
(type mn -h for details)
The mn utility creates Mininet network from the command line. It can create
parametrized topologies, invoke the Mininet CLI, and run tests.
Options:
-h, --help                show this help message and exit
--switch=SWITCH            default|ivs|lxb|ovs|ovsbr|ovsk[user[,param=value...]]
                           user=UsersSwitch ovs=OVSSwitch ovsbr=OVSBridge
                           ovsk=OVSSwitch ivs=IVSSwitch lxb=LinuxBridge
                           default=OVSSwitch
--host=HOST                cfs|proc|rt[,param=value...] proc=Host
                           rt=CPULimitedHost['sched': 'rt']
                           cfs=CPULimitedHost['sched': 'cfs']
--controller=CONTROLLER  default|none|nox|ovsc|ref|remote|ryu[,param=value...]
                           ref=Controller ovsc=OVSController nox=NOX
                           remote=RemoteController ryu=Ryu
                           default=DefaultController none=NullController
--link=LINK                default|ovs|tc|tcu[,param=value...] default=Link
                           tc=TCLink tcu=TCULink ovs=OVSLink
--topo=TOPO                linear|minimal|reversed|single|torus|tree[,param=value
                           ...] minimal=MinimalTopo linear=LinearTopo
                           reversed=SingleSwitchReversedTopo
                           single=SingleSwitchTopo tree=TreeTopo torus=TorusTopo
-c, --clean                clean and exit
--custom=CUSTOM            read custom classes or params from .py file(s)
--test=TEST                pingall|pingpair|iperf|iperfudp|all|none|build
-x, --xterms                spawn xterms for each node
-i IPBASE, --ipbase=IPBASE base IP address for hosts
--mac                      automatically set host MACs
--arp                      set all-pairs ARP entries
-v VERBOSITY, --verbosity=VERBOSITY debug|info|output|warning|warn|error|critical
```

```
Jan 22 05:57
ubuntu@ubuntu: ~
--nolistenport            don't use passive listening port
--pre=PRE                 CLI script to run before tests
--post=POST               CLI script to run after tests
--pin                     pin hosts to CPU cores (requires --host cfs or --host
                           rt)
--nat                     [option=val...] adds a NAT to the topology that
                           connects Mininet hosts to the physical network.
                           Warning: This may route any traffic on the machine
                           that uses Mininet's IP subnet into the Mininet
                           network. If you need to change Mininet's IP subnet,
                           see the --ipbase option.
--version                 prints the version and exits
-w, --wait                wait for switches to connect
-t WAIT, --twait=WAIT    timed wait (s) for switches to connect
--cluster=server1,server2... run on multiple servers (experimental!)
--placement=block|random node placement for --cluster (experimental!)
```

Phase 2: Setting Up the SDN Controller (OpenDaylight) We deployed the "brain" of the network using the OpenDaylight (ODL) Titanium release.

Download open daylight titanium zip in kali first.



1. **Installing Java Runtime:** Since the newer versions of OpenDaylight require a modern Java environment, we installed OpenJDK 21.

- o **Command:** `sudo apt install openjdk-21-jdk -y`

```
karaf-0.22.1 must be version 21.0 or greater
ubuntu@ubuntu:~/karaf-0.22.1$ sudo apt install openjdk-21-jdk -y
[sudo] password for ubuntu:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  openjdk-21-jdk-headless openjdk-21-jre openjdk-21-jre-headless
Suggested packages:
  openjdk-21-demo openjdk-21-source visualvm fonts-ipafont-gothic fonts-ipafont-mincho
```

2. **Deploying the Controller:** We downloaded the ODL distribution, extracted it, and launched the Karaf container.

- **Commands:**

Bash

`unzip karaf-0.22.1.zip`

`cd karaf-0.22.1`

`./bin/karaf`

2. **Enabling Controller Features:** Inside the Karaf console, we installed the plugins required for the REST API (to talk to Python) and OpenFlow (to talk to switches).

- **Command:** `feature:install odl-restconf odl-openflowplugin-flow-services-rest`

ubuntu [Running] - Oracle VirtualBox

File Machine View Input Devices Help

Jan 22 07:40

ubuntu@ubuntu: ~/Downloads

```
update-alternatives: using /usr/lib/jvm/java-11-openjdk-amd64/bin/rmic to provide /usr/bin/rmic (rmic) in auto mode
update-alternatives: using /usr/lib/jvm/java-11-openjdk-amd64/bin/serialver to provide /usr/bin/serialver (serialver) in auto mode
update-alternatives: using /usr/lib/jvm/java-11-openjdk-amd64/bin/jaotc to provide /usr/bin/jaotc (jaotc) in auto mode
update-alternatives: using /usr/lib/jvm/java-11-openjdk-amd64/bin/jhsdb to provide /usr/bin/jhsdb (jhsdb) in auto mode
Setting up openjdk-11-jdk:amd64 (11.0.29+7-1ubuntu1-24.04) ...
update-alternatives: using /usr/lib/jvm/java-11-openjdk-amd64/bin/jconsole to provide /usr/bin/jconsole (jconsole) in auto mode
ubuntu@ubuntu:~$ cd downloads
bash: cd: downloads: No such file or directory
ubuntu@ubuntu:~$ cd Downloads
ubuntu@ubuntu:~/Downloads$ unzip karaf-*.zip
Archive: karaf-0.22.1.zip
  inflating: karaf-0.22.1/CONTRIBUTING.md
  inflating: karaf-0.22.1/LICENSE
  creating: karaf-0.22.1/system/
  creating: karaf-0.22.1/system/tech/
  creating: karaf-0.22.1/system/tech/pantheon/
  creating: karaf-0.22.1/system/tech/pantheon/triemap/
  creating: karaf-0.22.1/system/tech/pantheon/triemap/pt-triemap/
  creating: karaf-0.22.1/system/tech/pantheon/triemap/pt-triemap/1.3.2/
  inflating: karaf-0.22.1/system/tech/pantheon/triemap/pt-triemap/1.3.2/pt-triemap-1.3.2-features.xml
  creating: karaf-0.22.1/system/tech/pantheon/triemap/triemap/
  inflating: karaf-0.22.1/system/tech/pantheon/triemap/triemap/maven-metadata-local.xml
```

Jan 22 07:46

ubuntu@ubuntu: ~/karaf-0.22.1

```
creating: karaf-0.22.1/deploy/
inflating: karaf-0.22.1/deploy/README
creating: karaf-0.22.1/data/
creating: karaf-0.22.1/data/tmp/
inflating: karaf-0.22.1/data/tmp/README
ubuntu@ubuntu:~/Downloads$ mv karaf-* ~/
ubuntu@ubuntu:~/Downloads$ ~/karaf-0.22.1 > ./bin/karaf
bash: ./bin/karaf: No such file or directory
ubuntu@ubuntu:~/Downloads$ ~/karaf-0.22.1 > /bin/karaf
bash: /bin/karaf: Permission denied
ubuntu@ubuntu:~/Downloads$ cd karaf-0.22.1
bash: cd: karaf-0.22.1: No such file or directory
ubuntu@ubuntu:~/Downloads$ cd ..
ubuntu@ubuntu:~$ cd karaf-0.22.1
ubuntu@ubuntu:~/karaf-0.22.1$ ./bin/karaf
karaf: JAVA_HOME not set; results may vary
karaf: JVM must be version 21 or greater
```


- **Command:**

Bash

`curl -u admin:admin http://localhost:8181/rests/data/network-topology:network-topology?content=nonconfig`

```

Jan 22 08:08
ubuntu@ubuntu: ~/karaf-0.22.1
After this operation, 534 kB of additional disk space will be used.
Ign:1 http://in.archive.ubuntu.com/ubuntu noble-updates/main amd64 curl amd64 8.5.0-2ubuntu10.6
0% [Working]^C
ubuntu@ubuntu:~/karaf-0.22.1$ sudo apt install curl
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following NEW packages will be installed:
  curl
0 upgraded, 1 newly installed, 0 to remove and 216 not upgraded.
Need to get 226 kB of archives.
After this operation, 534 kB of additional disk space will be used.
Get:1 http://in.archive.ubuntu.com/ubuntu noble-updates/main amd64 curl amd64 8.5.0-2ubuntu10.6 [226 kB]
Fetched 226 kB in 7s (32.1 kB/s)
Selecting previously unselected package curl.
(Reading database ... 152588 files and directories currently installed.)
Preparing to unpack .../curl_8.5.0-2ubuntu10.6_amd64.deb ...
Unpacking curl (8.5.0-2ubuntu10.6) ...
Setting up curl (8.5.0-2ubuntu10.6) ...
Processing triggers for man-db (2.12.0-4build2) ...
ubuntu@ubuntu:~/karaf-0.22.1$ curl -u admin:admin http://localhost:8181/rests/data/network-topology:network-topology ? c
ontent -nonconfig
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload  Total   Spent    Left     Speed
 100    77   100    77    0     0    33      0  0:00:02  0:00:02 --:--:--   33
curl: (3) URL rejected: No host part in the URL
curl: (6) Could not resolve host: content
ubuntu@ubuntu:~/karaf-0.22.1$ curl -u admin:admin http://localhost:8181/rests/data/network-topology:network-topology?con
tent=nonconfig
{"network-topology:network-topology":{"topology":[{"topology-id":"flow:1"}]}}ubuntu@ubuntu:~/karaf-0.22.1$

```

Phase 3: Linking Mininet to OpenDaylight This was the critical integration step where we connected the data plane (Mininet) to the control plane (OpenDaylight).

1. **Launching the Topology & Establishing the Link:** We started Mininet with a custom topology (1 switch, 3 hosts). Crucially, we used the `--controller=remote` flag to link the switch to our OpenDaylight controller running on 127.0.0.1 (localhost).

- **Command:**

```

sudo mn --controller=remote,ip=127.0.0.1,port=6633 --topo=single,3 --
switch=ovsk,protocols=OpenFlow13

```

```

Jan 22 08:08
ubuntu@ubuntu: ~/karaf-0.22.1
ubuntu@ubuntu: ~/karaf-0.22.1
ubuntu@ubuntu: ~/karaf-0.22.1
ubuntu@ubuntu: ~/karaf-0.22.1$ sudo mn --controller=remote,ip=127.0.0.1,port=6633 --topo=single,3 --switch=ovsk,protocols
=OpenFlow13
[sudo] password for ubuntu:
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2 h3
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1) (h3, s1)
*** Configuring hosts
h1 h2 h3
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Starting CLI:
mininet>

```

```

curl: (6) Could not resolve host: content
ubuntu@ubuntu: ~/karaf-0.22.1$ curl -u admin:admin http://localhost:8181/rests/data/network-topology:network-topology?con
tent=nonconfig
{"network-topology:network-topology":{"topology":[{"topology-id":"flow:1"}]}
ubuntu@ubuntu: ~/karaf-0.22.1$ curl -u admin
:admin http://localhost:8181/rests/data/network-topology:network-topology?content=nonconfig
{"network-topology:network-topology":{"topology":[{"topology-id":"flow:1","node":[{"node-id":"openflow:1","opendaylight-
topology-inventory:inventory-node-ref":"/opendaylight-inventory:nodes/node[id='openflow:1']","termination-point":[{"tp-i
d":"openflow:1:LOCAL","opendaylight-topology-inventory:inventory-node-connector-ref":"/opendaylight-inventory:nodes/node
[id='openflow:1']/node-connector[id='openflow:1:LOCAL']"},{"tp-id":"openflow:1:1","opendaylight-topology-inventory:inven
tory-node-connector-ref":"/opendaylight-inventory:nodes/node[id='openflow:1']/node-connector[id='openflow:1:1']"},{"tp-i
d":"openflow:1:2","opendaylight-topology-inventory:inventory-node-connector-ref":"/opendaylight-inventory:nodes/node[id=
'openflow:1']/node-connector[id='openflow:1:2']"},{"tp-id":"openflow:1:3","opendaylight-topology-inventory:inventory-nod
e-connector-ref":"/opendaylight-inventory:nodes/node[id='openflow:1']/node-connector[id='openflow:1:3']"}]}]}]}
ubuntu@ubuntu: ~/karaf-0.22.1$

```

Establishing Baseline Connectivity: By default, an SDN switch drops all packets until told otherwise. To verify the link was active, we manually pushed a "Normal" flow rule to Switch 1 to allow basic traffic.

- **Command:** `sudo ovs-ofctl add-flow s1 action=normal -O OpenFlow13`

```

d":"openflow:1:2","opendaylight-topology-inventory:inventory-node-connector-ref":"/opendaylight-inventory:nodes/node[i
'openflow:1']/node-connector[id='openflow:1:2']"},{"tp-id":"openflow:1:3","opendaylight-topology-inventory:inventory-n
e-connector-ref":"/opendaylight-inventory:nodes/node[id='openflow:1']/node-connector[id='openflow:1:3']"}]}]}]}
ubuntu@ubuntu: ~/karaf-0.22.1$ sudo ovs-ofctl add-flow s1 action=normal -O OpenFlow13
ubuntu@ubuntu: ~/karaf-0.22.1$

```

```

h1 h2 h3
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Starting CLI:
mininet> h1 ping h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=0.453 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.049 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.050 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.168 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0.071 ms
^C
--- 10.0.0.2 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4114ms
rtt min/avg/max/mdev = 0.049/0.158/0.453/0.153 ms
mininet>

```

Phase 4: Middleware Configuration (Python) We implemented a Python script to bridge the controller's data with our database.

1. **Setting up the Environment:** We created a virtual environment to manage dependencies safely.

- o **Commands:**

```
sudo apt install python3-venv -y
```

```
python3 -m venv sdn_env
```

```
source sdn_env/bin/activate
```

```
pip install requests prometheus-client
```

```

ubuntu@ubuntu:~/karaf-0.22.1$ mkdir ~/sdn-pp
ubuntu@ubuntu:~/karaf-0.22.1$ cd ~/sdn-pp
ubuntu@ubuntu:~/sdn-pp$ source sdn.env/bin/activate
bash: sdn.env/bin/activate: No such file or directory
ubuntu@ubuntu:~/sdn-pp$ source sdn_env/bin/activate
bash: sdn_env/bin/activate: No such file or directory
ubuntu@ubuntu:~/sdn-pp$ sudo apt install python3-venv -y
[sudo] password for ubuntu:
Reading package lists... Done
Building dependency tree... Done

```

```

Setting up python3-venv (3.12.3-0ubuntu2.1) ...
ubuntu@ubuntu:~/sdn-pp$ python3 -m venv sdn_env
ubuntu@ubuntu:~/sdn-pp$ source sdn_env/bin/activate
(sdn_env) ubuntu@ubuntu:~/sdn-pp$ pip install requests prometheus-client
Collecting requests
  Downloading requests-2.32.5-py3-none-any.whl.metadata (4.9 kB)
Collecting prometheus-client
  Downloading prometheus_client-0.24.1-py3-none-any.whl.metadata (2.1 kB)

```

```
Installing collected packages: urllib3, prometheus-client, idna, cha
Successfully installed certifi-2026.1.4 charset_normalizer-3.4.4 idn
ib3-2.6.3
(sdn_env) ubuntu@ubuntu:~/sdn-pp$ nano sdn_exporter.py
(sdn_env) ubuntu@ubuntu:~/sdn-pp$ python3 sdn_exporter.py
```

Copy paste code .

```
import time
```

```
import requests
```

```
from prometheus_client import start_http_server, Gauge
```

```
# Define the metric
```

```
PACKETS_TX = Gauge('sdn_port_packets_transmitted', 'Packets sent per port',
['switch_id', 'port_id'])
```

```
def fetch_metrics():
```

```
    # Trying the most common URL for your version
```

```
    url = "http://localhost:8181/rests/data/opendaylight-
inventory:nodes?content=nonconfig"
```

```
    auth = ('admin', 'admin')
```

```
    try:
```

```
        r = requests.get(url, auth=auth, headers={'Accept': 'application/json'})
```

```
        if r.status_code == 200:
```

```
            data = r.json()
```

```
            # This logic searches for any 'node' entry in the data
```

```
            nodes = data.get('opendaylight-inventory:nodes', {}).get('node', []) or
data.get('nodes', {}).get('node', [])
```

```
            for node in nodes:
```

```
                s_id = node.get('id', 'unknown')
```

```
                connectors = node.get('node-connector', [])
```

```

for conn in connectors:

    p_id = conn.get('id', 'unknown')

    # Search for the statistics block specifically

    # Some versions use different prefixes; this covers the most common ones

    stats = conn.get('opendaylight-port-statistics:flow-capable-node-connector-
statistics', {})

    if not stats:

        stats = conn.get('flow-capable-node-connector-statistics', {})

    packets = stats.get('packets', {}).get('transmitted', 0)

    # Update Prometheus

    PACKETS_TX.labels(switch_id=s_id, port_id=p_id).set(packets)

    print(f"Updated: {s_id} Port {p_id} -> {packets} packets")

else:

    print(f"ODL Error: {r.status_code}")

except Exception as e:

    print(f"Script Error: {e}")

if __name__ == '__main__':

    start_http_server(8000)

    print("Exporter is live on http://localhost:8000")

    while True:

        fetch_metrics()

        time.sleep(2)

```

```
Jan 23 03:07
ubuntu@ubuntu: ~/sdn-pp
GNU nano 7.2 sdn_exporter.py
import time
import requests
from prometheus_client import start_http_server, Gauge

# Define the metric
PACKETS_TX = Gauge('sdn_port_packets_transmitted', 'Packets sent per port', ['switch_id', 'port_id'])

def fetch_metrics():
    # Trying the most common URL for your version
    url = "http://localhost:8181/rests/data/opendaylight-inventory:nodes?content=nonconfig"
    auth = ('admin', 'admin')
    try:
        r = requests.get(url, auth=auth, headers={'Accept': 'application/json'})
        if r.status_code == 200:
            data = r.json()
            # This logic searches for any 'node' entry in the data
            nodes = data.get('opendaylight-inventory:nodes', {}).get('node', []) or data.get('nodes', {}).get('node', [])
            for node in nodes:
                s_id = node.get('id', 'unknown')
                connectors = node.get('node-connector', [])
                for conn in connectors:
                    p_id = conn.get('id', 'unknown')
                    # Search for the statistics block specifically
                    # Some versions use different prefixes; this covers the most common ones
                    # Read 46 lines
    except:
        pass

^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo      M-A Set Mark
^X Exit      ^R Read File  ^_ Replace    ^U Paste      ^J Justify    ^/ Go To Line  M-E Redo      M-6 Copy
```

```
Jan 23 03:09
ubuntu@ubuntu: ~/sdn-pp
ubuntu@ubuntu:~$ cd sdn-pp
ubuntu@ubuntu:~/sdn-pp$ source sdn_env/bin/activate
(sdn_env) ubuntu@ubuntu:~/sdn-pp$ python3 sdn_exporter.py
^CTraceback (most recent call last):
  File "/home/ubuntu/sdn-pp/sdn_exporter.py", line 29, in <module>
    time.sleep(2)
KeyboardInterrupt

(sdn_env) ubuntu@ubuntu:~/sdn-pp$ nano sdn_exporter.py
(sdn_env) ubuntu@ubuntu:~/sdn-pp$ python3 sdn_exporter.py
Exporter is live on http://localhost:8000
Updated: openflow:1 Port openflow:1:3 -> 922 packets
Updated: openflow:1 Port openflow:1:LOCAL -> 0 packets
Updated: openflow:1 Port openflow:1:2 -> 3123 packets
Updated: openflow:1 Port openflow:1:1 -> 3123 packets
Updated: openflow:1 Port openflow:1:3 -> 923 packets
Updated: openflow:1 Port openflow:1:LOCAL -> 0 packets
Updated: openflow:1 Port openflow:1:2 -> 3127 packets
Updated: openflow:1 Port openflow:1:1 -> 3127 packets
Updated: openflow:1 Port openflow:1:3 -> 923 packets
Updated: openflow:1 Port openflow:1:LOCAL -> 0 packets
Updated: openflow:1 Port openflow:1:2 -> 3127 packets
Updated: openflow:1 Port openflow:1:1 -> 3127 packets
Updated: openflow:1 Port openflow:1:3 -> 923 packets
Updated: openflow:1 Port openflow:1:LOCAL -> 0 packets
Updated: openflow:1 Port openflow:1:2 -> 3130 packets
Updated: openflow:1 Port openflow:1:1 -> 3130 packets
Updated: openflow:1 Port openflow:1:3 -> 924 packets
Updated: openflow:1 Port openflow:1:LOCAL -> 0 packets
Updated: openflow:1 Port openflow:1:2 -> 3134 packets
Updated: openflow:1 Port openflow:1:1 -> 3134 packets
```

In browser <http://localhost:8000>


```
ubuntu@ubuntu: ~/prometheus-2.55.0.linux-amd64$ nano prometheus.yml
# my global config
global:
  scrape_interval: 15s # Set the scrape interval to every 15 seconds. Default is every 1
  minute.
  evaluation_interval: 15s # Evaluate rules every 15 seconds. The default is every 1
  minute.
  # scrape_timeout is set to the global default (10s).

# Alertmanager configuration
alerting:
  alertmanagers:
    - static_configs:
      - targets:
        # - alertmanager:9093

# Load rules once and periodically evaluate them according to the global 'evaluation_interval'.
rule_files:
  # - "first_rules.yml"
  # - "second_rules.yml"

# A scrape configuration containing exactly one endpoint to scrape:
# Here it's Prometheus itself.
scrape_configs:
  - job_name: 'prometheus'
    static_configs:
      - targets: ['localhost:9090']
    # The job name is added as a label `job=<job_name>` to any timeseries scraped from this config.
  - job_name: 'sdn_monitoring'

    # metrics_path defaults to '/metrics'
    # scheme defaults to 'http'.

    static_configs:
      - targets: ["localhost:8000"]
```

my global config

global:

scrape_interval: 15s # Set the scrape interval to every 15 seconds. Default is every 1 minute.

evaluation_interval: 15s # Evaluate rules every 15 seconds. The default is every 1 minute.

scrape_timeout is set to the global default (10s).

Alertmanager configuration

alerting:

alertmanagers:

- static_configs:

- targets:

- # - alertmanager:9093

```
# Load rules once and periodically evaluate them according to the global
'evaluation_interval'.
```

```
rule_files:
```

```
# - "first_rules.yml"
```

```
# - "second_rules.yml"
```

```
# A scrape configuration containing exactly one endpoint to scrape:
```

```
# Here it's Prometheus itself.
```

```
scrape_configs:
```

```
# The job name is added as a label `job=<job_name>` to any timeseries scraped from
this config.
```

```
- job_name: "prometheus"
```

```
# metrics_path defaults to '/metrics'
```

```
# scheme defaults to 'http'.
```

```
static_configs:
```

```
- targets: ["localhost:9090"]
```

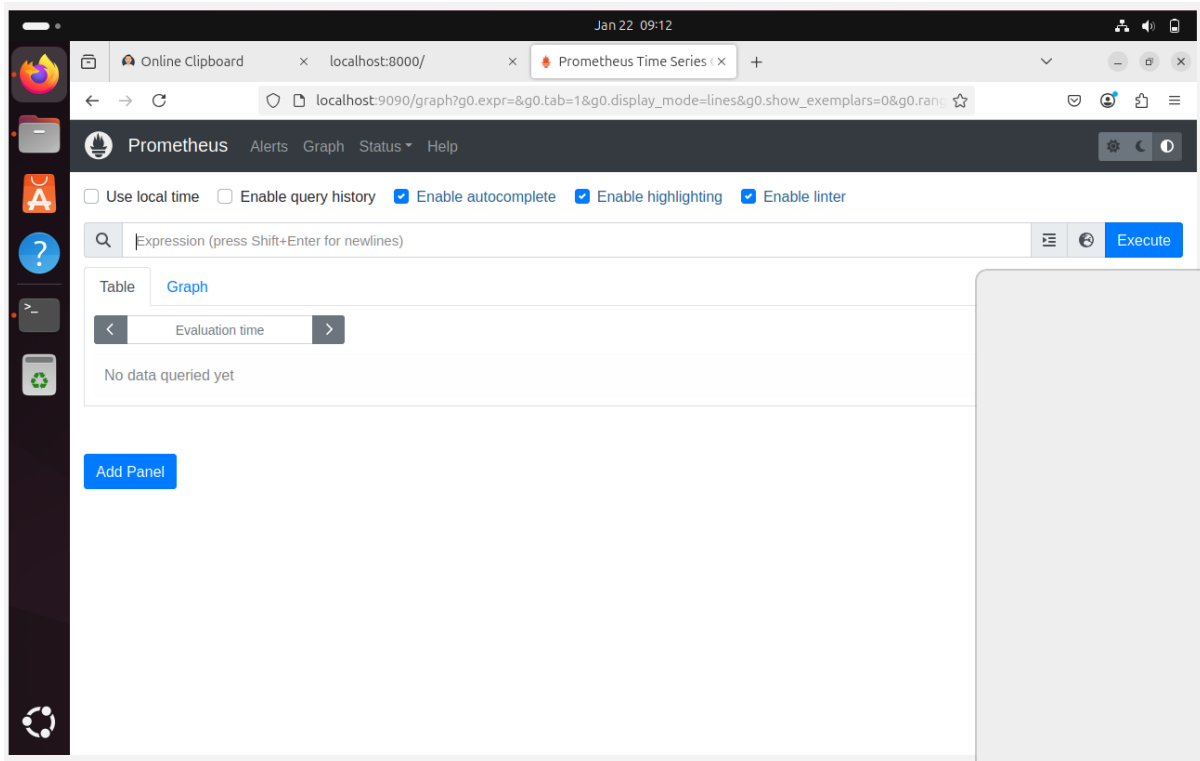
```
- job_name: 'sdn_monitoring'
```

```
static_configs:
```

```
- targets: ['localhost:8000']
```

```
ubuntu@ubuntu:~/prometheus-2.55.0.linux-amd64$ ./prometheus --config.file=prometheus.yml
ts=2026-01-22T09:10:59.274Z caller=main.go:627 level=info msg="No time or size retention was set so using the default time retention" duration=15d
ts=2026-01-22T09:10:59.283Z caller=main.go:671 level=info msg="Starting Prometheus Server" mode=server version="(version=2.55.0, branch=HEAD, revision=91d80252c3e528728b0f88d254dd720f6be07cb8)"
ts=2026-01-22T09:10:59.283Z caller=main.go:676 level=info build_context="(go=go1.23.2, platform=linux/amd64, compiler=gc, build_date=20241022-13:47:22, tags=netgo,builtinassets,stringlabels)"
ts=2026-01-22T09:10:59.283Z caller=main.go:677 level=info host_details="(Linux 6.14.0-37-generic #37-Ubuntu SMP PREEMPT_DYNAMIC Thu Nov 20 10:25:38 UTC 2 x86_64 ubuntu (none))"
ts=2026-01-22T09:10:59.283Z caller=main.go:678 level=info fd_limits="(soft=1048576, hard=1048576)"
ts=2026-01-22T09:10:59.283Z caller=main.go:679 level=info vm_limits="(soft=unlimited, hard=unlimited)"
ts=2026-01-22T09:10:59.285Z caller=web.go:585 level=info component=web msg="Start listening for connections"
ts=2026-01-22T09:10:59.285Z caller=main.go:1197 level=info msg="Starting TSDB ..."
ts=2026-01-22T09:10:59.298Z caller=head.go:627 level=info component=tsdb msg="Replaying on-disk memory index"
ts=2026-01-22T09:10:59.298Z caller=head.go:714 level=info component=tsdb msg="On-disk memory index loaded"
ts=2026-01-22T09:10:59.298Z caller=head.go:722 level=info component=tsdb msg="Replaying WAL, this may take a while"
ts=2026-01-22T09:10:59.300Z caller=tls_config.go:348 level=info component=web msg="Listening on"
ts=2026-01-22T09:10:59.301Z caller=tls_config.go:351 level=info component=web msg="TLS is disabled"
ts=2026-01-22T09:10:59.301Z caller=head.go:794 level=info component=tsdb msg="WAL segment loaded"
ts=2026-01-22T09:10:59.301Z caller=head.go:831 level=info component=tsdb msg="WAL replay completed"
ts=2026-01-22T09:10:59.303Z caller=main.go:1218 level=info fs_type=EXT4_SUPER_MAGIC
ts=2026-01-22T09:10:59.304Z caller=main.go:1221 level=info msg="TSDB started"
```

Go to <http://localhost:9090>



Go to status

Go to target

Jan 22 09:31

Online Clipboard x localhost:8000/ x Prometheus Time Series x

localhost:9090/targets?search=

Prometheus Alerts Graph Status Help

Targets

All scrape pools All Unhealthy Collapse All Filter by endpoint or labels Unknown Unhealthy Healthy

prometheus (1/1 up) show less

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://localhost:9090/metrics	UP	instance="localhost:9090" job="prometheus"	2.923s ago	6.410ms	

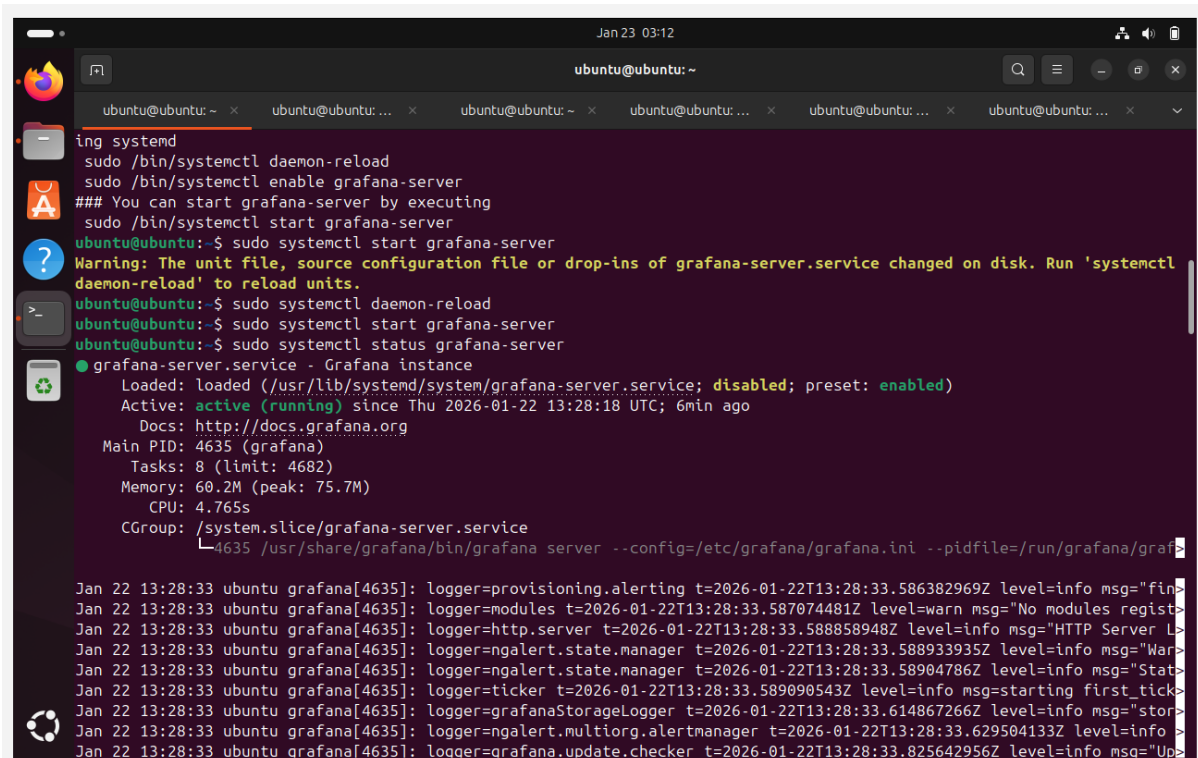
sdn_monitoring (1/1 up) show less

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://localhost:8000/metrics	UP	instance="localhost:8000" job="sdn_monitoring"	10.160s ago	2.087ms	

Jan 23 03:11

ubuntu@ubuntu: ~

```
ubuntu@ubuntu:~$ sudo apt-get install -y adduser libfontconfig1 musl
wget https://dl.grafana.com/oss/release/grafana_10.0.3_amd64.deb
sudo dpkg -i grafana_10.0.3_amd64.deb
sudo systemctl start grafana-server
[sudo] password for ubuntu:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
adduser is already the newest version (3.137ubuntu1).
adduser set to manually installed.
libfontconfig1 is already the newest version (2.15.0-1.1ubuntu2).
libfontconfig1 set to manually installed.
The following NEW packages will be installed:
  musl
0 upgraded, 1 newly installed, 0 to remove and 216 not upgraded.
Need to get 416 kB of archives.
After this operation, 804 kB of additional disk space will be used.
Get:1 http://in.archive.ubuntu.com/ubuntu noble/universe amd64 musl amd64 1.2.4-2 [416 kB]
Fetched 416 kB in 1s (334 kB/s)
Selecting previously unselected package musl:amd64.
(Reading database ... 152637 files and directories currently installed.)
Preparing to unpack .../musl_1.2.4-2_amd64.deb ...
Unpacking musl:amd64 (1.2.4-2) ...
Setting up musl:amd64 (1.2.4-2) ...
Processing triggers for man-db (2.12.0-4build2) ...
--2026-01-22 13:19:21-- https://dl.grafana.com/oss/release/grafana_10.0.3_amd64.deb
Resolving dl.grafana.com (dl.grafana.com)... 199.232.170.217, 2a04:4e42:6a::729
Connecting to dl.grafana.com (dl.grafana.com)|199.232.170.217|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 83428792 (80M) [application/octet-stream]
Saving to: 'grafana_10.0.3_amd64.deb'
```

A terminal window on Ubuntu showing the installation and startup of Grafana. The user runs 'sudo systemctl daemon-reload', 'sudo systemctl enable grafana-server', and 'sudo systemctl start grafana-server'. A warning message appears: 'Warning: The unit file, source configuration file or drop-ins of grafana-server.service changed on disk. Run 'systemctl daemon-reload' to reload units.' The user then runs 'sudo systemctl daemon-reload' and 'sudo systemctl status grafana-server'. The status output shows that the service is loaded, active (running), and has a preset of 'enabled'. It also displays the main PID, tasks, memory, CPU, and CGroup information. At the bottom, there are several log messages from the Grafana service, including provisioning, modules, http server, ngalert, ticker, and storage logs.

```
ing systemd
sudo /bin/systemctl daemon-reload
sudo /bin/systemctl enable grafana-server
### You can start grafana-server by executing
sudo /bin/systemctl start grafana-server
ubuntu@ubuntu:~$ sudo systemctl start grafana-server
Warning: The unit file, source configuration file or drop-ins of grafana-server.service changed on disk. Run 'systemctl
daemon-reload' to reload units.
ubuntu@ubuntu:~$ sudo systemctl daemon-reload
ubuntu@ubuntu:~$ sudo systemctl start grafana-server
ubuntu@ubuntu:~$ sudo systemctl status grafana-server
● grafana-server.service - Grafana instance
   Loaded: loaded (/usr/lib/systemd/system/grafana-server.service; disabled; preset: enabled)
   Active: active (running) since Thu 2026-01-22 13:28:18 UTC; 6min ago
     Docs: http://docs.grafana.org
    Main PID: 4635 (grafana)
      Tasks: 8 (limit: 4682)
     Memory: 60.2M (peak: 75.7M)
        CPU: 4.765s
       CGroup: /system.slice/grafana-server.service
              └─4635 /usr/share/grafana/bin/grafana server --config=/etc/grafana/grafana.ini --pidfile=/run/grafana/graf>

Jan 22 13:28:33 ubuntu grafana[4635]: logger=provisioning.alerting t=2026-01-22T13:28:33.586382969Z level=info msg="fin>
Jan 22 13:28:33 ubuntu grafana[4635]: logger=modules t=2026-01-22T13:28:33.587074481Z level=warn msg="No modules regist>
Jan 22 13:28:33 ubuntu grafana[4635]: logger=http.server t=2026-01-22T13:28:33.588858948Z level=info msg="HTTP Server L>
Jan 22 13:28:33 ubuntu grafana[4635]: logger=ngalert.state.manager t=2026-01-22T13:28:33.588933935Z level=info msg="War>
Jan 22 13:28:33 ubuntu grafana[4635]: logger=ngalert.state.manager t=2026-01-22T13:28:33.58904786Z level=info msg="Stat>
Jan 22 13:28:33 ubuntu grafana[4635]: logger=ticker t=2026-01-22T13:28:33.589090543Z level=info msg="starting first_tick>
Jan 22 13:28:33 ubuntu grafana[4635]: logger=grafanaStorageLogger t=2026-01-22T13:28:33.614867266Z level=info msg="stor>
Jan 22 13:28:33 ubuntu grafana[4635]: logger=ngalert.multiorg.alertmanager t=2026-01-22T13:28:33.629504133Z level=info >
Jan 22 13:28:33 ubuntu grafana[4635]: logger=grafana.update.checker t=2026-01-22T13:28:33.825642956Z level=info msg="Up>
```

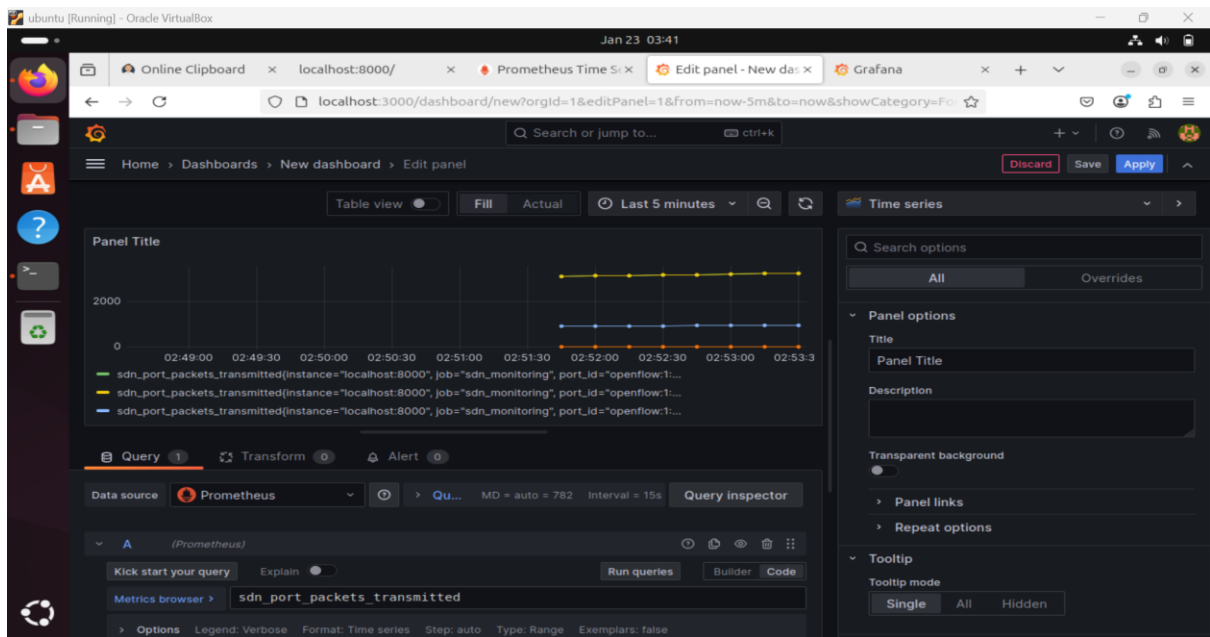
Go to Grafana login with admin and admin.

Add Data Source:

- Click **Connections** -> **Data Sources** -> **Add new data source**.
- Select **Prometheus**.
- URL: `http://localhost:9090` (This points to your Prometheus server).
- Click **Save & Test**.

Create the Dashboard:

- Click **Dashboards** -> **New Dashboard** -> **Add Visualization**.
- Select the Prometheus data source you just added.
- In the query box, type: `sdn_port_packets_transmitted`
- Click **Run Queries**.



```
^Cmininet> h2 iperf -s -u &
mininet> h1 iperf -u -c 10.0.0.2 -b 50M -t 30

Client connecting to 10.0.0.2, UDP port 5001
Sending 1470 byte datagrams, IPG target: 224.30 us (kalman adjust)
UDP buffer size: 208 KByte (default)

[ 1] local 10.0.0.1 port 41658 connected with 10.0.0.2 port 5001
[ ID] Interval      Transfer    Bandwidth
[ 1] 0.0000-30.0002 sec 188 MBytes  52.4 Mbits/sec
[ 1] Sent 133751 datagrams
[ 1] Server Report:
[ ID] Interval      Transfer    Bandwidth      Jitter    Lost/Total Datagrams
[ 1] 0.0000-29.9997 sec 187 MBytes  52.4 Mbits/sec  0.003 ms 160/133750 (0.12%)
mininet>
```

